SUST_Ragebait_Tree

Shahjalal University of Science and Technology

1 Templates and Scripts 1
- 1.1 sublime . . . 1
- 1.2 vs code . . . 1
- 1.3 Interactor . . . 1
- 1.4 debug.h . . . 1
- 1.5 Template - Nahdi . . . 1
- 1.6 Template - Ibnul . . . 1
- 1.7 Mod Template . . . 2

2 Number Theory 2
- 2.1 Binomial Coeffecient . . . 2
- 2.2 Sieve . . . 2
- 2.3 CRT . . . 2

3 Data Structure 2
- 3.1 Fenwick Tree . . . 2
- 3.2 Segment Tree . . . 2
- 3.3 DSU . . . 3
- 3.4 Persistent Segment Tree . . . 3

4 Graph 3
- 4.1 Dijkstra . . . 3
- 4.2 LCA . . . 3
- 4.3 SCC . . . 4
- 4.4 HLD . . . 4
- 4.5 Centroid Decomposition . . . 4

5 String 4
- 5.1 Trie . . . 4
- 5.2 Z Algorithm . . . 4

6 Geometry 4
- 6.1 Common . . . 4
- 6.2 Vector2d . . . 4
- 6.3 Vector3d . . . 5

7 Miscellaneous 5
- 7.1 Generationg Permutations . . . 5
- 7.2 Iterating over Submasks . . . 5
- 7.3 Random . . . 5
- 7.4 pbds . . . 5

8 Math 5
- 8.1 Equations . . . 5
- 8.2 Recurrences . . . 5
- 8.3 Trigonometry . . . 5
- 8.4 Geometry . . . 5
- 8.5 Derivatives/Integrals . . . 6
- 8.6 Sums . . . 6
- 8.7 Series . . . 6
- 8.8 Probability theory . . . 6
- 8.9 Markov chains . . . 6
- 8.10 Number Theory . . . 7

9 Combinatorial 7
- 9.1 Permutations . . . 7
- 9.2 Partitions and subsets . . . 7
- 9.3 General purpose numbers . . . 7

1 Templates and Scripts

1.1 sublime

```
{
  "shell_cmd": "g++ -Wall -std=c++20 -O2
     -DLOCAL $file -o $file_base_name
    && bash -c 'ulimit -S -f 10000;
    timeout 5 ./$file_base_name < input
    .in > output.out 2> error.log'",
  "file_regex": "^(..[^:]*):([0-9]+)
     :?([0-9]+)?:? (.*)$",
  "working_dir": "${file_path}",
  "selector": "source.c++, source.c,
     source.cpp",
  "variants":
  [
    {
      "name": "sanitize",
      "shell_cmd": "g++ -Wall -std=c++20
      -fsanitize=address -fsanitize=
    undefined -DLOCAL $file -o
    $file_base_name && bash -c 'ulimit
    -S -f 10000; timeout 5 ./
    $file_base_name < input.in > output
    .out 2> error.log'"
    }
  ]
}
```

1.2 vs code

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "cp",
      "type": "shell",
      "command": "",
      "args": [
        "g++",
        "-Wall",
        "-std=c++20",
        "-O2",
        "-DLOCAL",
        "\"${file}\"",
        "-o",
        "\"${fileDirname}/${
    fileBasenameNoExtension}\"",
        "&&",
        "\"${fileDirname}/${
    fileBasenameNoExtension}\"",
        "<",
        "input.in",
        ">",
        "output.out",
        "2>",
        "error.log"
      ],
      "group": "build",
      "presentation": {
        "reveal": "silent"
      },
      "problemMatcher": {
        "owner": "cpp",
        "fileLocation": ["relative", "${
    workspaceRoot}"],
        "pattern": {
          "regexp": "^(.*):(\\d+):(\\d+)
    :\\s+(warning|error):\\s+(.*)$",
          "file": 1,
          "line": 2,
          "column": 3,
          "severity": 4,
          "message": 5
        }
      }
    }
  ]
}
```

1.3 Interactor

```
socat EXEC:"./interactor" EXEC:"./soln"
```

1.4 debug.h

```
#include <iostream>
using namespace std;
#define see(x) cerr << #x << ": " << x
    << nl
template <typename... Args>
void debug(Args... args) {
  ((cerr << " " << args), ...) << "\n";
}
```

1.5 Template - Nahdi

```
#include <bits/stdc++.h>
using namespace std;
using vi = vector<int>;
using vii = vector<vi>;
using vs = vector<string>;
using ii = pair<int, int>;
using vpi = vector<ii>;
using ll = long long;
using vll = vector<ll>;
using pll = pair<ll, ll>;
#define f first
#define s second
#define forn(i, n) for (int i = 0; i < n
    ; i++)
#define fore(i, a, b) for (int i = a; i
    <= b; i++)
#define for1n(i, n) for (int i = 1; i <=
     n; i++)
#define all(x) x.begin(), x.end()
#define Flag cout << "Reached here.\n"
#define FASTIO              \
  ios::sync_with_stdio(0); \
  cin.tie(0);              \
  cout.tie(0);
#define pb push_back
#define pbb pop_back
#define sz size
#define rsz resize
#define ins insert
#define MAX 1000000000ll
#define MIN -MAX
#define MOD 1000000007LL
#define clr(x, y) memset(x, y, sizeof(x)
    )
int dx[] = {0, 1, -1, 0};
int dy[] = {1, 0, 0, -1};
void init() {
#ifndef ONLINE_JUDGE
  freopen("output.txt", "w", stdout);
  freopen("input.txt", "r", stdin);
#endif
}
void solve() {}
int main() {
  // init();
  // FASTIO;
  int t;
  cin >> t;
  while (t--) solve();
}
```

1.6 Template - Ibnul

```
#include <bits/stdc++.h>
using namespace std;
// Template
//
    ====================================

// pbds
// #include <ext/pb_ds/assoc_container.
    hpp>
// #include <ext/pb_ds/tree_policy.hpp>
// using namespace __gnu_pbds;
// template<typename T, typename comp =
    less<T>>
// using ordered_set =  tree<T,
    null_type, comp, rb_tree_tag,
// tree_order_statistics_node_update>;
// Debugging
#ifdef LOCAL
#include "debug.h"
#else
#define debug(...)
#define see(x)
#endif
typedef long long ll;
typedef vector<int> VI;
typedef vector<long long> VLL;
typedef vector<bool> VB;
typedef vector<vector<int>> VVI;
typedef pair<int, int> PI;
typedef pair<ll, ll> PLL;
```

```
typedef vector<pair<int, int>> VPI;
#define pb push_back
#define ff first
#define ss second
#define mp make_pair
#define all(a) a.begin(), a.end()
#define revall(a) a.rbegin(), a.rend()
#define loop(i, s, e) for (int i = s; i
    < e; ++i)
#define inp(v) \
  for (auto& x : v) cin >> x
#define outp(v, st) \
  for (int i = st, n = v.size(); i < n;
    ++i) cout << v[i] << " \n"[i == n -
     1]
#define nl "\n"
#define yep cout << "YES\n"
#define nope cout << "NO\n"
#define INF (int)1e9
#define INFL (ll)1e18
#define EPS 1e-9
#define MOD 998244353
// #define MOD 1000000007
#define MAXN 2002
//
    ==================================

void solve(int tc) {}
int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(NULL);
  int t = 1;
  // cin >> t;
  for (int i = 1; i <= t; ++i) solve(i);
#ifdef LOCAL
  cerr << "Execution time: " << 1000.f *
      clock() / CLOCKS_PER_SEC << " ms."
       << nl;
#endif
  return 0;
}
```

1.7 Mod Template

```
template <unsigned P>
struct modint {
  unsigned v;
  modint() = default;
  template <class T>
  modint(T x) {
    x %= (signed)P, v = x < 0 ? x + P :
     x;
  }
  modint operator+() const { return *
     this; }
  modint operator-() const { return
     modint(0) - *this; }
  static modint power(modint a, int b) {
    if (b < 0) return power(a, -b).inv()
     ;
    modint res = 1;
    while (b) {
      if (b & 1) res *= a;
      a *= a;
      b >>= 1;
    }
    return res;
  }
  modint inv() const { return assert(v),
      power(*this, P - 2); }
  modint& operator+=(const modint& q) {
    (v += q.v) < P || (v -= P);
    return *this;
  }
  modint& operator-=(const modint& q) {
    (v -= q.v) < P || (v += P);
    return *this;
  }
  modint& operator*=(const modint& q) {
    v = 1ull * v * q.v % P;
    return *this;
  }
  modint& operator/=(const modint& q) {
    v = 1ull * v * q.inv().v % P;
    return *this;
  }
  friend modint operator+(modint p,
     const modint& q) { return p += q; }
  friend modint operator-(modint p,
     const modint& q) { return p -= q; }
  friend modint operator*(modint p,
     const modint& q) { return p *= q; }
  friend modint operator/(modint p,
     const modint& q) { return p /= q; }
  friend istream& operator>>(istream& is
     , modint& a) {
    int x;
    is >> x;
    x %= (signed)P;
    a.v = x < 0 ? x + P : x;
    return is;
  }
  friend ostream& operator<<(ostream& os
     , const modint& a) {
    os << a.v;
    return os;
  }
};
typedef modint<998244353> mint;
typedef modint<1000000007> mint2;
```

2 Number Theory

2.1 Binomial Coeffecient

```
ll fact[MAXN];
void calc_fact(ll mod = MOD) {
  fact[0] = 1;
  for (int i = 1; i < MAXN; ++i) {
    fact[i] = (fact[i - 1] * i) % mod;
  }
}
ll C(int n, int k, ll mod = MOD) {
  if (k > n) return 0;
  return (1LL * fact[n] * inv((1LL *
     fact[k] * fact[n - k]) % mod)) %
     mod;
}
```

2.2 Sieve

```
VI primes;
bool isprime[MAXN];
void sieve() {
  isprime[1] = false;
  for (int i = 2; i < MAXN; ++i) isprime
     [i] = true;
  for (int i = 2; i < MAXN; ++i) {
    if (!isprime[i]) continue;
    primes.pb(i);
    if (1LL * i * i >= MAXN) continue;
    for (int j = i * i; j < MAXN; j += i
     ) isprime[j] = false;
  }
}
```

2.3 CRT

```
int extended_gcd(ll a, ll b, ll& x, ll&
     y) {
  if (!b) {
    x = 1, y = 0;
    return a;
  }
  ll g = extended_gcd(b, a % b, x, y);
  swap(x, y);
  y -= x * (a / b);
  return g;
}
ll modinv(ll a, ll m) {
  ll x, y;
  extended_gcd(a, m, x, y);
  x %= m;
  return x < 0 ? x + m : x;
}
ll crt(vector<pair<ll, ll>>& mods) {
  ll mod = 1;
  for (auto& p : mods) mod *= p.ss;
  ll ans = 0;
  for (auto [x, p] : mods) {
    ll m = mod / p;
    m = (m * modinv(m, p)) % mod;
    m = (m * x) % mod;
    ans = (ans + m) % mod;
  }
  return ans;
}
```

3 Data Structure

3.1 Fenwick Tree

```
struct fenwick {
  int n;
  vector<ll> tree;
  fenwick(int s) {
    n = s;
    tree.assign(n + 1, 0);
  }
  ll get(int i) {
    ll res = 0;
    for (; i > 0; i -= i & -i) res +=
     tree[i];
    return res;
  }
  void add(int i, ll x) {
    for (; i <= n; i += i & -i) tree[i]
     += x;
  }
  void add(int l, int r, ll x) {
    add(l, x);
    if (r < n) add(r + 1, -x);
  }
  ll get(int l, int r) { return get(r) -
      get(l - 1); }
};
```

3.2 Segment Tree

```
template <typename T>
struct segtree {
#define left(u) 2 * u + 1
#define right(u) 2 * u + 2
  int n;
  T init_val;
  vector<T> tree;
  function<T(T, T)> f;
  bool is_lazy = false;
  T init_lazy;
  vector<T> lazy;
  function<void(T&, T, int, int)>
     apply_lazy;
  function<void(T&, T)> merge_lazy;
  segtree() {}  // Dummy constructor for
      global declaration
  segtree(int sz, function<T(T, T)> f, T
      val = 0) {
    n = sz;
    this->f = f;
    init_val = val;
    tree.assign(1 << (__lg(n - 1) + 2),
     init_val);
  }
  // apply(tree[u], lazy[u], l, r)
  // merge(lazy[left(u)], lazy[u])
  void make_lazy(function<void(T&, T,
     int, int)> apply,
                 function<void(T&, T)>
     merge, T init = 0) {
    is_lazy = true;
    init_lazy = init;
    apply_lazy = apply;
    merge_lazy = merge;
    lazy.assign(tree.size(), init_lazy);
  }
  void build(vector<T>& a) { _build(0,
     0, n - 1, a); }
  void update(int i, T x) { _update(0,
     0, n - 1, i, x); }
  void update(int l, int r, T x) {
     _update_range(0, 0, n - 1, l, r, x)
     ; }
  T get(int l, int r) { return _get(0,
     0, n - 1, l, r); }
  void _build(int u, int l, int r,
     vector<T>& a) {
    if (l == r) {
      tree[u] = a[l];
      return;
    }
    int mid = (l + r) / 2;
    _build(left(u), l, mid, a);
    _build(right(u), mid + 1, r, a);
```

```
    tree[u] = f(tree[left(u)], tree[
     right(u)]);
  }
  void _update(int u, int l, int r, int
     i, T x) {
    if (l == r) {
      tree[u] = x;
      return;
    }
    int mid = (l + r) / 2;
    if (i <= mid)
      _update(left(u), l, mid, i, x);
    else
      _update(right(u), mid + 1, r, i, x
     );
    tree[u] = f(tree[left(u)], tree[
     right(u)]);
  }
  void propagate(int u, int l, int r) {
    if (lazy[u] == init_lazy || l == r)
     return;
    int m = (l + r) / 2;
    apply_lazy(tree[left(u)], lazy[u], l
     , m);
    merge_lazy(lazy[left(u)], lazy[u]);
    apply_lazy(tree[right(u)], lazy[u],
     m + 1, r);
    merge_lazy(lazy[right(u)], lazy[u]);
    lazy[u] = init_lazy;
  }
  void _update_range(int u, int tl, int
     tr, int l, int r, T x) {
    if (l > tr || r < tl) return;
    if (tl >= l && tr <= r) {
      apply_lazy(tree[u], x, tl, tr);
      merge_lazy(lazy[u], x);
      return;
    }
    propagate(u, tl, tr);
    int mid = (tl + tr) / 2;
    _update_range(left(u), tl, mid, l, r
     , x);
    _update_range(right(u), mid + 1, tr,
      l, r, x);
    tree[u] = f(tree[left(u)], tree[
     right(u)]);
  }
  T _get(int u, int tl, int tr, int l,
     int r) {
    if (l > tr || r < tl) return
     init_val;
    if (l <= tl && r >= tr) return tree[
     u];
    if (is_lazy) propagate(u, tl, tr);
    int mid = (tl + tr) / 2;
    return f(_get(left(u), tl, mid, l, r
     ), _get(right(u), mid + 1, tr, l, r
     ));
  }
#undef left
#undef right
};
```

3.3 DSU

```
struct dsu {
  vector<int> parent, size;
  dsu(int n) {
    parent.resize(n + 1);
    size.resize(n + 1);
    for (int i = 1; i <= n; ++i) {
      parent[i] = i;
      size[i] = 1;
    }
  }
  int find(int u) {
    if (parent[u] == u) return u;
    return parent[u] = find(parent[u]);
  }
  void merge(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      if (size[u] < size[v]) swap(u, v);
      parent[v] = u;
      size[u] += size[v];
    }
  }
};
```

3.4 Persistent Segment Tree

```
template <typename T>
struct node {
  T val;
  int l, r;
};
template <typename T>
struct persistent_segtree {
  int n;
  T init_val;
  vector<node<T>> nodes;
  vector<int> version;
  function<T(T, T)> f;
  persistent_segtree() {}
  persistent_segtree(int sz, function<T(
     T, T)> f, T val = 0) {
    n = sz;
    this->f = f;
    init_val = val;
    build(vector<T>(n, init_val));
  }
  persistent_segtree(vector<T>& a,
     function<T(T, T)> f, T val = 0) {
    n = a.size();
    this->f = f;
    init_val = val;
    build(a);
  }
  int build(vector<T> a) {
    int root = _build(0, n - 1, a);
    version.pb(root);
    return version.size() - 1;
  }
  int update(int i, T x, int v = -1) {
    if (v == -1) v = version.size() - 1;
    int root = _update(version[v], 0, n
     - 1, i, x);
    version.pb(root);
    return version.size() - 1;
  }
  T get(int l, int r, int v) { return
     _get(version[v], 0, n - 1, l, r); }
  // version is 1 indexed (0 is the
     initial before updates)
  T get_kth(int ver_l, int ver_r, int k)
     {
    return _get_kth(version[ver_l - 1],
     version[ver_r], 0, n - 1, k);
  }
  int _build(int l, int r, vector<T>& a)
     {
    int root = nodes.size();
    nodes.pb({});
    if (l == r) {
      nodes[root].val = a[l];
      return root;
    }
    int mid = (l + r) / 2;
    int new_child = _build(l, mid, a);
    nodes[root].l = new_child;
    new_child = _build(mid + 1, r, a);
    nodes[root].r = new_child;
    nodes[root].val = f(nodes[nodes[root
     ].l].val, nodes[nodes[root].r].val)
     ;
    return root;
  }
  int _update(int u, int l, int r, int i
     , T x) {
    int root = nodes.size();
    node<T> old = nodes[u];
    nodes.pb(old);
    if (l == r) {
      nodes[root].val = x;
      return root;
    }
    int mid = (l + r) / 2;
    if (i <= mid) {
      int newl = _update(nodes[u].l, l,
     mid, i, x);
      nodes[root].l = newl;
    } else {
      int newr = _update(nodes[u].r, mid
     + 1, r, i, x);
      nodes[root].r = newr;
    }
    nodes[root].val = f(nodes[nodes[root
     ].l].val, nodes[nodes[root].r].val)
     ;
    return root;
  }
  T _get(int u, int tl, int tr, int l,
     int r) {
    if (l > tr || r < tl) return
     init_val;
    if (l <= tl && r >= tr) return nodes
     [u].val;
    int mid = (tl + tr) / 2;
    return f(_get(nodes[u].l, tl, mid, l
     , r),
             _get(nodes[u].r, mid + 1,
     tr, l, r));
  }
  int _get_kth(int u, int v, int l, int
     r, int k) {
    if (l == r) return l;
    int mid = (l + r) / 2;
    int kl = nodes[nodes[v].l].val -
     nodes[nodes[u].l].val;
    if (kl >= k)
      return _get_kth(nodes[u].l, nodes[
     v].l, l, mid, k);
    else
      return _get_kth(nodes[u].r, nodes[
     v].r, mid + 1, r, k - kl);
  }
};
```

4 Graph

4.1 Dijkstra

```
struct Node {
  int u;
  ll dist;
  bool operator<(const Node& v) const {
     return dist > v.dist; }
};
vector<ll> dijkstra(int start, vector<
     VPI>& adj) {
  vector<ll> dis(adj.size(), INFL);
  priority_queue<Node> q;
  dis[start] = 0;
  q.push({start, 0});
  while (!q.empty()) {
    Node node = q.top();
    int u = node.u;
    q.pop();
    if (dis[u] < node.dist) continue;
    for (auto e : adj[u]) {
      if (node.dist + e.ss < dis[e.ff])
     {
        dis[e.ff] = node.dist + e.ss;
        q.push({e.ff, dis[e.ff]});
      }
    }
  }
  return dis;
}
```

4.2 LCA

```
struct LCA {
  int step = 0;
  VI vis_start, vis_end;
  VVI st;
  int k;
  VVI& tree;
  LCA(VVI& tree) : tree(tree) { init(
     tree.size() - 1); }
  void build_st(int u, int p) {
    st[u][0] = p;
    for (int i = 1; i <= k; ++i) st[u][i
     ] = st[st[u][i - 1]][i - 1];
  }
  void dfs(int u, int p) {
    vis_start[u] = step++;
    build_st(u, p);
    for (auto v : tree[u]) {
      if (v != p) dfs(v, u);
    }
    vis_end[u] = step++;
  }
```

```
  bool is_ancestor(int a, int b) {
    return vis_start[a] < vis_start[b]
     && vis_end[a] > vis_end[b];
  }
  int get(int a, int b) {
    if (a == b) return a;
    if (is_ancestor(a, b)) return a;
    if (is_ancestor(b, a)) return b;
    for (int i = k; i >= 0; --i) {
      if (!is_ancestor(st[a][i], b)) a =
      st[a][i];
    }
    return st[a][0];
  }
  void init(int n) {
    vis_start.assign(n + 1, 0);
    vis_end.assign(n + 1, 0);
    k = __lg(2 * n - 1);
    st.assign(n + 1, VI(k + 1));
    dfs(1, 1);
  }
  // Extra
  int get_ancestor(int u, int n) {
    for (int i = 0; n > 0; ++i) {
      if (n & (1 << i)) {
        u = st[u][i];
        n -= (1 << i);
      }
    }
    return u;
  }
};
```

4.3 SCC

```
VVI scc;
VI stk;
int disc[MAXN], low[MAXN];
bool instk[MAXN];
int cur_time;
void dfs_scc(int u, VVI& g) {
  disc[u] = low[u] = ++cur_time;
  instk[u] = true;
  stk.pb(u);
  for (int v : g[u]) {
    if (!disc[v]) {
      dfs_scc(v, g);
      low[u] = min(low[u], low[v]);
    } else if (instk[v])
      low[u] = min(low[u], disc[v]);
  }
  if (low[u] < disc[u]) return;
  scc.pb(VI());
  bool rem = true;
  while (rem) {
    int v = stk.back();
    stk.pop_back();
    instk[v] = false;
    scc.back().pb(v);
    rem = v != u;
  }
}
void tarjan(VVI& g) {
  cur_time = 0;
  int n = g.size();
  for (int i = 0; i < n; ++i) disc[i] =
     0;
  for (int u = 1; u < n; ++u)
    if (!disc[u]) dfs_scc(u, g);
}
```

4.4 HLD

```
VI heavy, chain, label, par, sz, dep;
int timer;
void dfsz(int u, int p, VVI& tree) {
  sz[u] = 1;
  par[u] = p;
  dep[u] = dep[p] + 1;
  int mx = -1;
  for (int v : tree[u]) {
    if (v == p) continue;
    dfsz(v, u, tree);
    sz[u] += sz[v];
    if (sz[v] > mx) mx = sz[v], heavy[u]
      = v;
  }
}
void dfshld(int u, int p, VVI& tree, int
      head) {
  label[u] = timer++;
  chain[u] = head;
  if (heavy[u] != -1) dfshld(heavy[u], u
     , tree, head);
  for (int v : tree[u]) {
    if (v == p || v == heavy[u])
     continue;
    dfshld(v, u, tree, v);
  }
}
void inithld(int n, VVI& tree) {
  heavy = VI(n + 1, -1);
  chain = VI(n + 1);
  label = VI(n + 1);
  sz = VI(n + 1);
  par = VI(n + 1);
  dep = VI(n + 1);
  timer = 1;
  dfsz(1, 0, tree);
  dfshld(1, 0, tree, 1);
}
ll query(int u, int v, fenwick& bit) {
  ll ret = 0;  // init value
  for (; chain[u] != chain[v]; v = par[
     chain[v]]) {
    if (dep[chain[u]] > dep[chain[v]])
     swap(u, v);
    // ret ?= bit.rsq(label[chain[v]],
     label[v]);
  }
  if (dep[u] > dep[v]) swap(u, v);
  // lca = u
  // ret ?= bit.rsq(label[u], label[v]);
  return ret;
}
```

4.5 Centroid Decomposition

```
struct CD {
  vector<set<int>> tree;
  vector<int> par, sz;
  CD(vector<vector<int>>& tree) {
    for (auto& v : tree) {
      this->tree.emplace_back(v.begin(),
      v.end());
    }
    par.resize(tree.size());
    sz.resize(tree.size());
    build(1, 0);
  }
  void build(int u, int p) {
    dfs(u, p);
    int centroid = dfs(u, p, sz[u]);
    par[centroid] = p;
    vector<int> adj(tree[centroid].begin
     (), tree[centroid].end());
    for (int v : adj) {
      tree[v].erase(centroid);
      tree[centroid].erase(v);
      build(v, centroid);
    }
  }
  void dfs(int u, int p) {
    sz[u] = 1;
    for (int v : tree[u]) {
      if (v == p) continue;
      dfs(v, u);
      sz[u] += sz[v];
    }
  }
  int dfs(int u, int p, int n) {
    for (int v : tree[u]) {
      if (v == p) continue;
      if (sz[v] > n / 2) return dfs(v, u
     , n);
    }
    return u;
  }
  int operator[](int i) { return par[i];
     }
};
```

5 String

5.1 Trie

```
struct trienode {
  bool endmark;
  vector<int> child;
  trienode(int sz) {
    endmark = false;
    child.resize(sz, 0);
  }
};
struct trie {
  int sz;
  char fst;
  vector<trienode> nodes;
  trie(int alpha_sz, char alpha_start) {
    sz = alpha_sz;
    fst = alpha_start;
    // root at idx 0
    nodes.pb(trienode(sz));
  }
  void insert(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst]) {
        nodes[cur].child[c - fst] =
     nodes.size();
        nodes.pb(trienode(sz));
      }
      cur = nodes[cur].child[c - fst];
    }
    nodes[cur].endmark = true;
  }
  bool search(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst])
     return false;
      cur = nodes[cur].child[c - fst];
    }
    return nodes[cur].endmark;
  }
  bool erase(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst])
     return false;
      cur = nodes[cur].child[c - fst];
    }
    if (!nodes[cur].endmark) return
     false;
    nodes[cur].endmark = false;
    return true;
  }
};
```

5.2 Z Algorithm

```
VI Z(string& s) {
  int n = s.size();
  VI z(n);
  int l = 0, r = 0;
  for (int i = 0; i < n; ++i) {
    if (i <= r) z[i] = min(z[i - l], r -
      i + 1);
    while (i + z[i] < n && s[z[i]] == s[
     i + z[i]]) l = i, r = i + z[i]++;
  }
  return z;
}
```

6 Geometry

6.1 Common

```
#define EPS 1e-12
#define pi 3.14159265358979311599796 34\
  6854418516159057617187 5L
bool feq(const long double& a, const
     long double& b) {
  long double del = fabsl(a - b);
  if (del <= EPS) return true;
  return del <= EPS * max(fabsl(a),
     fabsl(b));
}
```

6.2 Vector2d

```
struct vector2d {
  long double x, y;
  bool operator==(const vector2d& p)
     const {
    return feq(x, p.x) && feq(y, p.y);
  }
  vector2d operator+(const vector2d& p)
     const {
    return {x + p.x, y + p.y};
  }
  vector2d operator-(const vector2d& p)
     const {
    return {x - p.x, y - p.y};
  }
  vector2d operator*(long double scalar)
      const {
    return {scalar * x, scalar * y};
  }
  friend vector2d operator*(long double
     s, const vector2d& v) {
    return {v.x * s, v.y * s};
  }
  vector2d operator/(long double scalar)
      const {
    return {x / scalar, y / scalar};
  }
  long double norm2() const {
    return x * x + y * y;
  }
  long double magnitude() const {
    return sqrtl(norm2());
  }
  vector2d unit() const {
    long double m = magnitude();
    if (feq(m, 0)) return {0, 0};
    return {x / m, y / m};
  }
  vector2d normal() const {
    vector2d v{y, -x};
    return v.unit();
  }
};
long double dist(const vector2d& p1,
     const vector2d& p2) {
  return (p1 - p2).magnitude();
}
long double dot(const vector2d& v1,
     const vector2d& v2) {
  return v1.x * v2.x + v1.y * v2.y;
}
long double angle(const vector2d& v1,
     const vector2d& v2) {
  long double m = v1.magnitude() * v2.
     magnitude();
  if (feq(m, 0)) return 0;
  long double v = dot(v1, v2) / m;
  return acosl(min(1.0L, max(-1.0L, v)))
     ;
}
long double cross(const vector2d& v1,
     const vector2d& v2) {
  return v1.x * v2.y - v1.y * v2.x;
}
```

6.3 Vector3d

```
struct vector3d {
  long double x, y, z;
  bool operator==(const vector3d& p)
     const {
    return feq(x, p.x) && feq(y, p.y) &&
      feq(z, p.z);
  }
  vector3d operator+(const vector3d& p)
     const {
    return {x + p.x, y + p.y, z + p.z};
  }
  vector3d operator-(const vector3d& p)
     const {
    return {x - p.x, y - p.y, z - p.z};
  }
  vector3d operator*(long double scalar)
      const {
    return {x * scalar, y * scalar, z *
     scalar};
  }
  friend vector3d operator*(long double
     s, const vector3d& v) {
    return {v.x * s, v.y * s, v.z * s};
  }
  vector3d operator/(long double scalar)
      const {
    return {x / scalar, y / scalar, z /
     scalar};
  }
  long double norm2() const {
    return x * x + y * y + z * z;
  }
  long double magnitude() const {
    return sqrtl(norm2());
  }
  vector3d unit() const {
    long double m = magnitude();
    if (feq(m, 0)) return {0, 0, 0};
    return *this / m;
  }
};
long double dist(const vector3d& p1,
     const vector3d& p2) {
  return (p1 - p2).magnitude();
}
long double dot(const vector3d& v1,
     const vector3d& v2) {
  return v1.x * v2.x + v1.y * v2.y + v1.
     z * v2.z;
}
long double angle(const vector3d& v1,
     const vector3d& v2) {
  long double m = v1.magnitude() * v2.
     magnitude();
  if (feq(m, 0)) return 0;
  long double v = dot(v1, v2) / m;
  return acosl(min(1.0L, max(-1.0L, v)))
     ;
}
vector3d cross(const vector3d& a, const
     vector3d& b) {
  return {
    a.y * b.z - a.z * b.y,
    a.z * b.x - a.x * b.z,
    a.x * b.y - a.y * b.x
  };
}
```

7 Miscellaneous

7.1 Generationg Permutations

```
VI a;
//...
sort(all(a));
do {
  // Process permutation...
} while (next_permutation(all(a)));
```

7.2 Iterating over Submasks

```
int n = 17;
for (int mask = 0; mask < (1 << n); ++
     mask) {
  for (int sub = mask; sub; sub = (sub -
      1) & mask) {
    // Process submask...
  }
}
```

7.3 Random

```
mt19937 rng(chrono::steady_clock::now().
     time_since_epoch().count());
int randint(int l, int r) {
  uniform_int_distribution<int> dist(l,
     r);
  return dist(rng);
}
```

7.4 pbds

```
// ordered set
ordered_set<int> st;
// ordered multiset
ordered_set<int, less_equal<int>> multi;
// insert element
st.insert(x);
// get index
st.order_of_key(x);
// access by index
st.find_by_order(idx);
```

8 Math

8.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

8.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

8.3 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\begin{aligned} &(V + W) \tan((v - w)/2) \\ &= (V - W) \tan((v + w)/2) \end{aligned}$$

where V, W are lengths of sides opposite angles v, w.

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

8.4 Geometry

8.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \dfrac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \dfrac{abc}{4A}$

Inradius: $r = \dfrac{A}{p}$

Length of median (divides triangle into two equal-area triangles):
$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

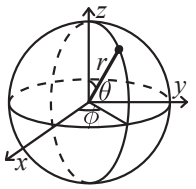
8.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180°, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

8.4.3 Spherical coordinates



$$x = r\sin\theta\cos\phi \qquad r = \sqrt{x^2+y^2+z^2}$$
$$y = r\sin\theta\sin\phi \qquad \theta = \arccos(z/r)$$
$$z = r\cos\theta \qquad \phi = \operatorname{atan2}(y,x)$$

8.5 Derivatives/Integrals

$$\tfrac{d}{dx}\arcsin x = \tfrac{1}{\sqrt{1-x^2}}$$
$$\tfrac{d}{dx}\arccos x = -\tfrac{1}{\sqrt{1-x^2}}$$
$$\tfrac{d}{dx}\tan x = 1 + \tan^2 x$$
$$\tfrac{d}{dx}\arctan x = \tfrac{1}{1+x^2}$$
$$\int \tan ax = -\tfrac{\ln|\cos ax|}{a}$$
$$\int x\sin ax = \tfrac{\sin ax - ax\cos ax}{a^2}$$
$$\int e^{-x^2} = \tfrac{\sqrt{\pi}}{2}\operatorname{erf}(x)$$
$$\int xe^{ax}dx = \tfrac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b fg' = fg\big|_a^b - \int_a^b f'g$$
$$\int u\,dv = uv - \int v\,du$$

8.6 Sums

$$\sum_{i=a}^{b} c^i = \frac{c^{b+1}-c^a}{c-1}, c \neq 1$$

$$\sum i = \tfrac{n(n+1)}{2}$$
$$\sum i^2 = \tfrac{n(n+1)(2n+1)}{6}$$
$$\sum i^3 = \tfrac{n^2(n+1)^2}{4}$$
$$\sum i^4 = \tfrac{n(n+1)(2n+1)}{30}(3n^2+3n-1)$$

8.7 Series

$$e^x = \sum_{n=0}^{\infty}\tfrac{x^n}{n!} \quad \text{all } x$$
$$\ln(1+x) = x - \tfrac{x^2}{2} + \tfrac{x^3}{3} - \dots \quad (-1 < x \leq 1)$$
$$\sin x = x - \tfrac{x^3}{3!} + \tfrac{x^5}{5!} - \dots \quad \text{all } x$$
$$\cos x = 1 - \tfrac{x^2}{2!} + \tfrac{x^4}{4!} - \dots \quad \text{all } x$$

8.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.
Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

8.8.1 Discrete distributions

8.8.2 Binomial Distribution

The number of successes in n independent yes/no experiments, each yielding success with probability p, is $\mathrm{Bin}(n,p)$.

$$P(X = k) = \binom{n}{k}p^k(1-p)^{n-k}$$

$$p(k) = \binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\mathrm{Bin}(n,p)$ is approximately $\mathrm{Po}(np)$ for small p.

8.8.2.0 First success distribution: The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\mathrm{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \dots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

8.8.2.0 Poisson distribution: The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\mathrm{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

8.8.3 Continuous distributions

8.8.3.0 Uniform distribution: If PDF is constant on $[a,b]$ and 0 elsewhere, it is $\mathrm{U}(a,b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

8.8.3.0 Exponential distribution: The time between events in a Poisson process is $\mathrm{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

8.8.3.0 Normal distribution: Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

8.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n\mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.
π is a stationary distribution if $\pi = \pi\mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i. π_j/π_i is the expected number of visits in state j between two visits in state i.
For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i's degree.
A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty}\mathbf{P}^k = \mathbf{1}\pi$.
A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j, is $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. The expected time until absorption, when the initial state is i, is $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$.

8.10 Number Theory

8.10.1 Euler's Totient (ϕ)

Counts integers $1 \le k \le n$ coprime to n.
$\phi(n) = n\prod_{p|n}(1-1/p)$.
If $\gcd(a,b)=1$, then $\phi(ab)=\phi(a)\phi(b)$.
For prime p, $\phi(p^k)=p^k-p^{k-1}$.
Property: $\sum_{d|n}\phi(d)=n$.

8.10.2 Euler's Theorem

If $\gcd(a,n)=1$, then $a^{\phi(n)} \equiv 1 \pmod n$.

8.10.2.0 Fermat's Little Theorem: For prime p and $p \nmid a$:
$a^{p-1} \equiv 1 \pmod p \implies a^{p-2} \equiv a^{-1} \pmod p$.

8.10.3 Wilson's Theorem

An integer $p > 1$ is prime if and only if:
$(p-1)! \equiv -1 \pmod p$.

8.10.4 Möbius Inversion

$$\mu(n) = \begin{cases} 0 & n \text{ not square-free} \\ 1 & \text{even \# prime factors} \\ -1 & \text{odd \# prime factors} \end{cases}$$

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n}\mu(d)g(n/d)$$

Other useful formulas/forms:
$\sum_{d|n}\mu(d) = [n=1]$ (very useful)
$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d}\mu(d/n)g(d)$
$g(n) = \sum_{1\le m\le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1\le m\le n}\mu(m)g(\lfloor \frac{n}{m} \rfloor)$

9 Combinatorial

9.1 Permutations

9.1.1 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

9.1.2 Derangements

Permutations where no element is in its original position.

$$\begin{aligned} D(n) &= (n-1)(D(n-1)+D(n-2)) \\ &= nD(n-1)+(-1)^n = \lfloor n!/e \rceil \end{aligned}$$

9.1.3 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$\begin{aligned} g(n) &= \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) \\ &= \frac{1}{n}\sum_{k|n} f(k)\phi(n/k) \end{aligned}$$

9.2 Partitions and subsets

9.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding order.

$$p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim \frac{0.145}{n}\exp(2.56\sqrt{n})$$

9.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

9.3 General purpose numbers

9.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).

$$B[0,\dots] = [1, -\tfrac{1}{2}, \tfrac{1}{6}, 0, -\tfrac{1}{30}, 0, \tfrac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k\cdot(n+1)^{m+1-k}$$

$$\begin{aligned} \sum_{i=m}^{\infty} f(i) &= \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m) \\ &\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} \\ &\quad + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

9.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$\begin{aligned} c(n,k) &= c(n-1,k-1)+ \\ &\quad (n-1)c(n-1,k) \\ \textstyle\sum_k c(n,k)x^k &= x(x+1)\dots(x+n-1) \end{aligned}$$

$c(8,k) = [0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1]$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, \dots$

9.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \ge j$, k j:s s.t. $\pi(j) > j$.

$$\begin{aligned} E_{n,k} &= (n-k)E_{n-1,k-1} + (k+1)E_{n-1,k} \\ E(n,0) &= E(n,n-1) = 1 \end{aligned}$$

$$E_{n,k} = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

9.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$\begin{aligned} S(n,k) &= S(n-1,k-1) + kS(n-1,k) \\ S(n,1) &= S(n,n) = 1 \\ S(n,k) &= \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n \end{aligned}$$

9.3.5 Bell numbers

Total partitions of n distinct elements.
$B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, \dots$

$$B(p^m+n) \equiv mB(n) + B(n+1) \pmod p$$

9.3.6 Labeled unrooted trees

- n vertices: n^{n-2}
- k trees of size n_i: $(\prod n_i)n^{k-2}$
- With degrees d_i: $\frac{(n-2)!}{\prod(d_i-1)!}$

9.3.7 Catalan numbers

$$C_n = \tfrac{1}{n+1}\tbinom{2n}{n} = \tbinom{2n}{n} - \tbinom{2n}{n+1} = \tfrac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n$$

$$C_{n+1} = \textstyle\sum_{i=0}^{n} C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.